

Intro2SE - Project Proposal - 4

PathToGrad

The student team is required to complete the **Project Proposal** documentation for the assigned course project, following the attached template.



Software Engineering Department
Faculty of Information and Technology
VNU-HCMUS

Table of Contents

Objectives	1
1 Member Contribution Assessment	2
2 Preliminary Problem Statement	3
3 Proposed Solution	4
4 Development Plan	6
5 Human Resources & Costing Plan	9
6 Tools setup	10
7 AI Usage Declaration	12
8 Presentation	13
9 Reflective Report	14

Project Proposal

Objectives

This document focus on the following topics:

- ✓ Completing the Project Proposal document with the following sections:
 - Preliminary Problem Statement
 - Proposed Solution
 - Development Plan
 - Human Resources & Costing Plan
- ✓ Understanding the Project Proposal document.
- ✓ This document will be used as input for AI tools to verify the quality of subsequent project artifacts.

All project artifacts must remain consistent and synchronized.

For example, if the project proposal is modified during this phase, a new version of the proposal must be documented.

By the conclusion of the project, all versions of every artifact must be submitted to demonstrate the evolution of your work.

1

Member Contribution Assessment

24127427 - Trần Nhật Đăng Khoa - 25%

- Task Evidence:

Task 1: Project Topic Selection and Coordination

Description: Facilitated discussion among team members regarding project topic selection. Gathered opinions, compared proposed ideas, evaluated feasibility, and guided the team toward selecting the final project topic, **PathToGrad - University Course Planning System**.

Task 2: Project Planning and Task Organization

Description: Defined the initial project structure and coordinated the distribution of responsibilities among team members. Established project roles, including **Project Manager, Business Analyst, Requirements Analyst, and System Architect**. Assigned proposal sections and responsibilities to ensure balanced workload distribution.

Task 3: Jira Project Setup

Description: Created and configured the **Jira project board**. Defined the sprint structure, created project tasks, assigned responsibilities, and established a workflow to track progress throughout the proposal phase.

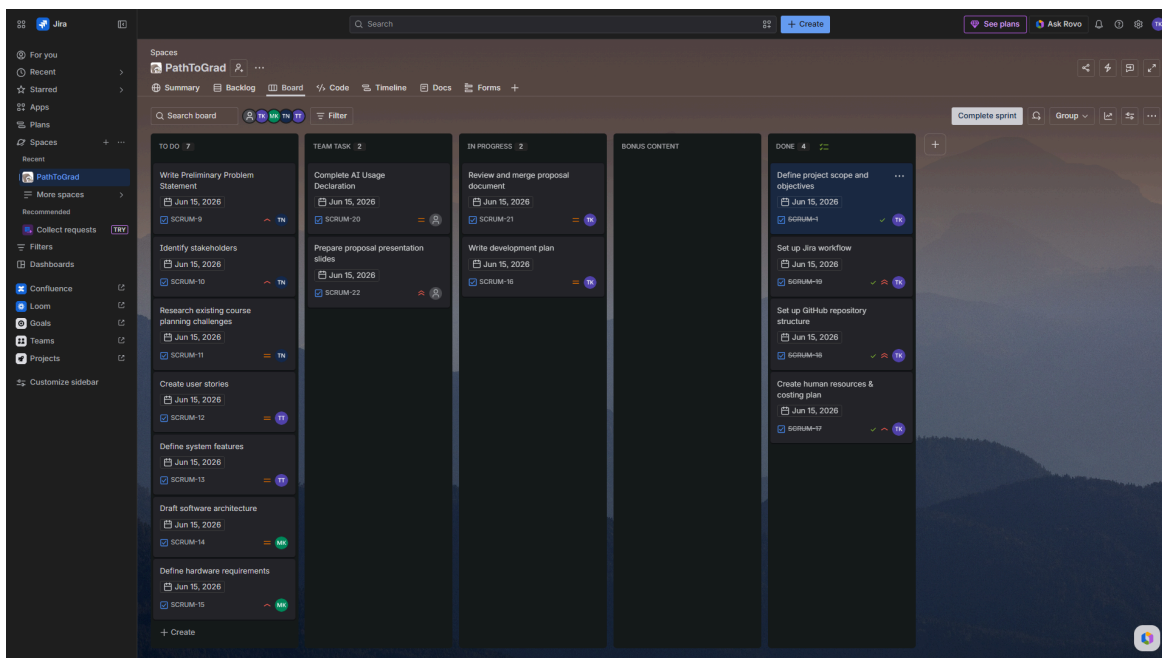
Task 4: GitHub Repository Setup

Description: Created and configured the project repository. Established the folder structure according to course requirements, including source code, documentation, testing, management, and assignment submission directories. Added repository documentation and organizational files.

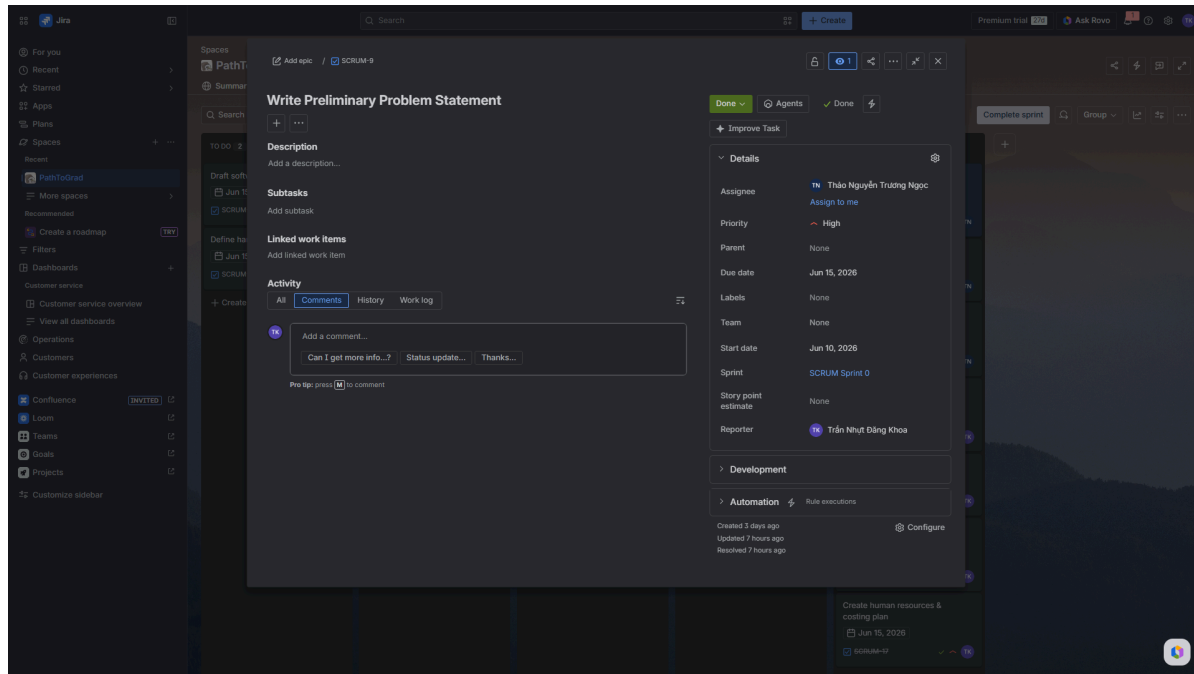
Task 5: Proposal Preparation and Documentation Support

Description: Prepared the project proposal environment by organizing project resources, creating documentation templates, reviewing project requirements, and coordinating team communication through **Discord**. Assisted team members with understanding proposal requirements and planning deliverables.

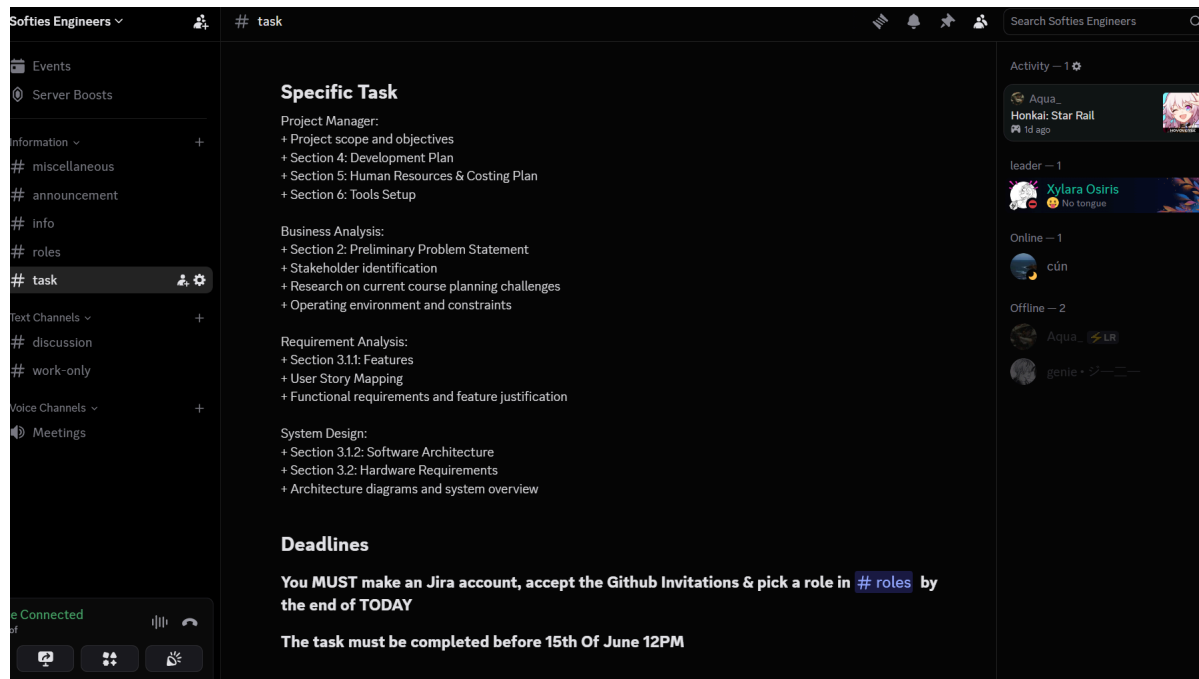
- **Visual Evidence:** Clear screenshots that explicitly show the Task Name, Assignee, Status, Assigned Date, and Completion Date.



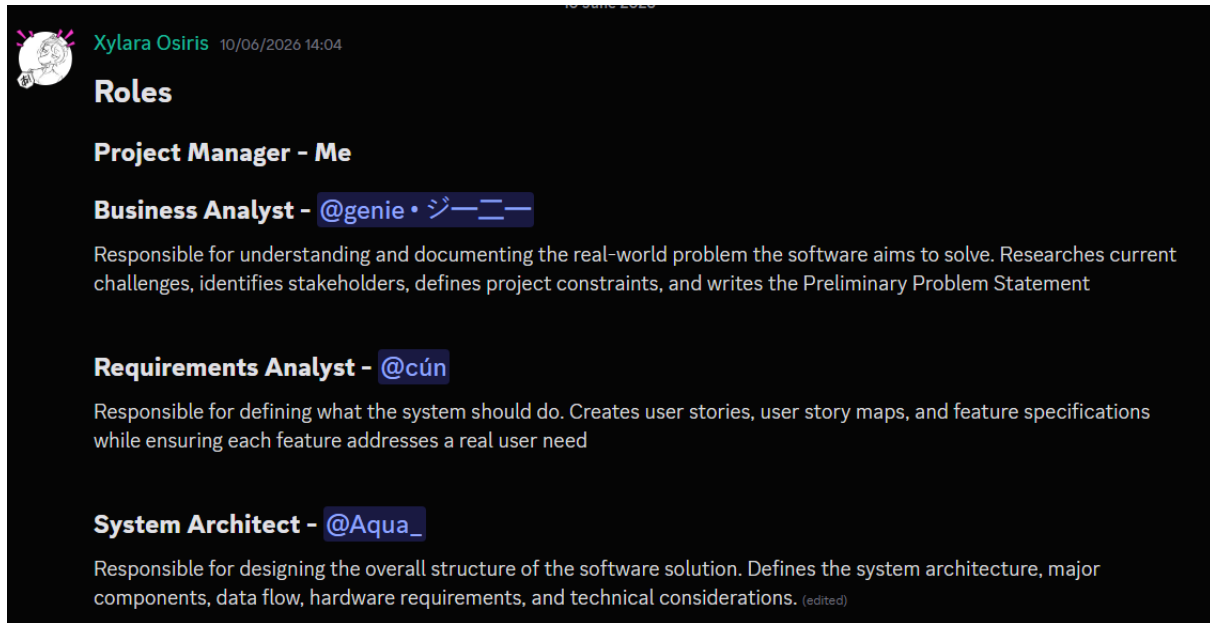
Assigned Tasks to different members on Jira



Detailed info on a task



Task specification on Discord



Splitting the roles and their description on Discord

24127543 - Nguyễn Trương Ngọc Thảo - 25%

- **Task Evidence:**

Task 1: Identify stakeholders

Description: Discover and categorize every person, group, or department that will interact with, build, or be affected by our software.

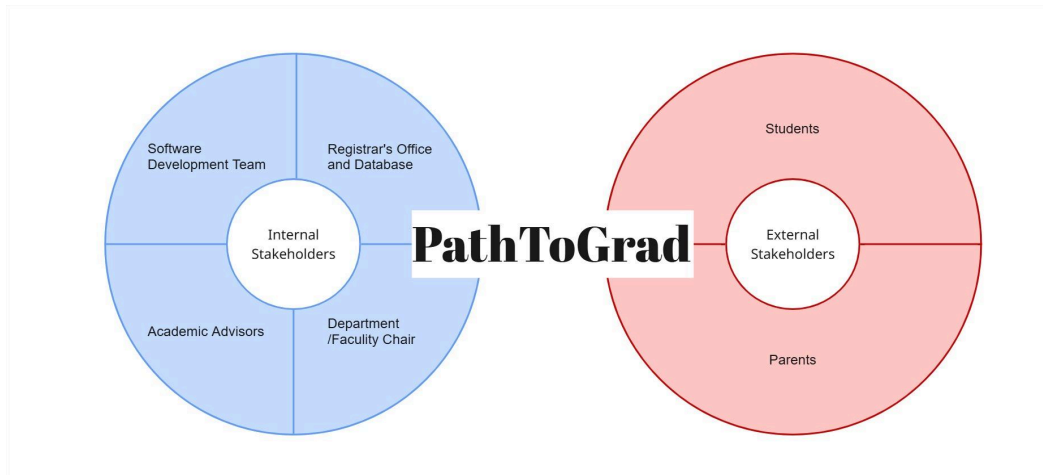
Task 2: Research existing course planning challenges

Description: Analyzing the pain points of the manual process: how students currently juggle scattered PDFs and disjointed catalogs, where human errors most frequently occur, and the negative consequences of those errors.

Task 3: Write Preliminary Problem Statement

Description: Defines the exact problem, who it affects, the negative impact it has on the university and students, and what a successful software solution looks like.

- **Visual Evidence:**



Stakeholders graph made on Miro

```
Existing course planning challenges?
- Course data on Moodle, requiring courses are in portals/PDF,...
Students have to look for information manually through them
-> Errors. Human makes mistakes. Manually tracking also leads to
misinformation(?), may miss out on compulsory courses

- Registration gets stuck when there's too many students accessing
at the same time. Timeline unclear (written as time-time) and hard
to imagine as a schedule. May get late if there's not enough
slots/other students are faster
-> It also helps the registra(?)

- Manually check points. Students dont know which course do they
have to study -> over study or under study led to undergraduate
and/or financial problems
```

Course planning challenges draft made on Notepad

- **Preliminary Problem Statement**

24127563 - Trần Thị Ngọc Trâm - 25%

- **Task Evidence:**

Task 1: Create User Stories

Description: Created the user stories for the **PathToGrad - University Course Planning System**. Identified the main stakeholders, created the user story map, wrote user stories with acceptance criteria, and linked each user story to related system features.

The user stories cover students, academic advisors, academic staff, the LLM Planning Agent, internal planning tools, and fallback planning mode.

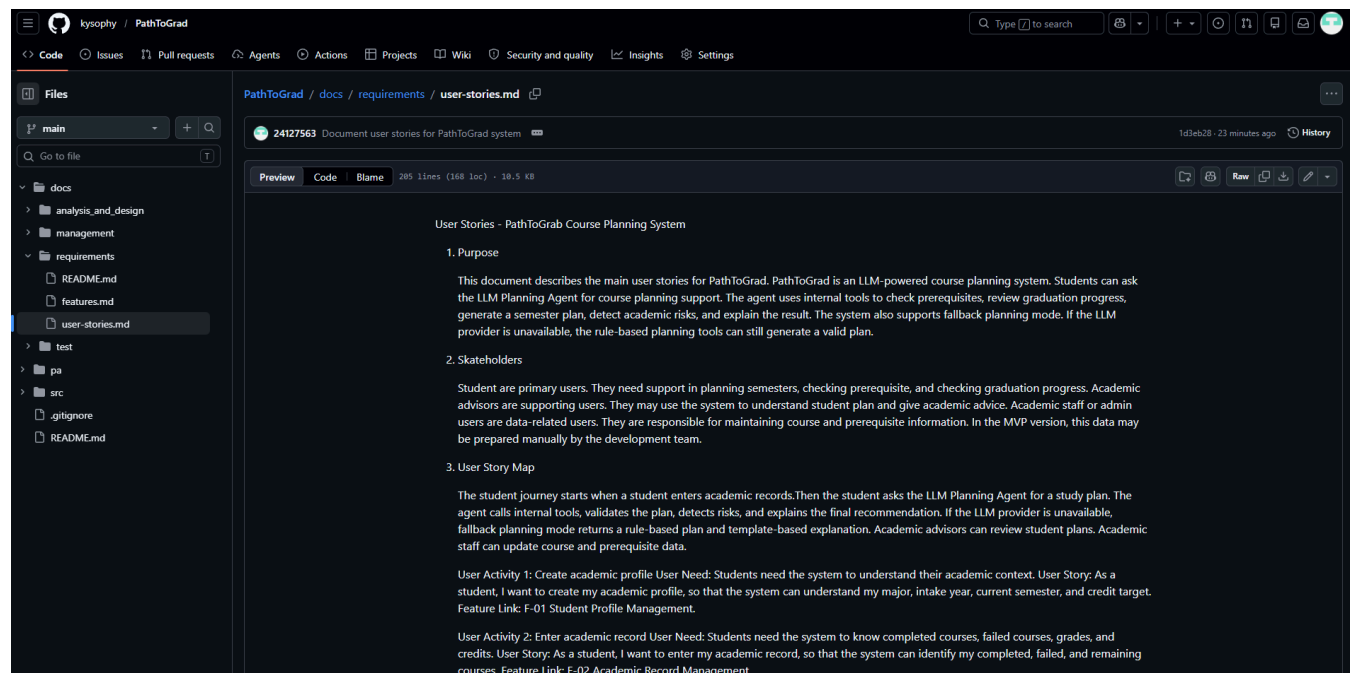
Task 2: Define System Features

Description: Defined the main system features of PathToGrad based on the user story map. Specified the LLM Planning Agent, internal tools, APIs, MCP-compatible tool interface, planning skills, fallback planning mode, functional requirements, non-functional requirements, constraints, and suggested technology stack.

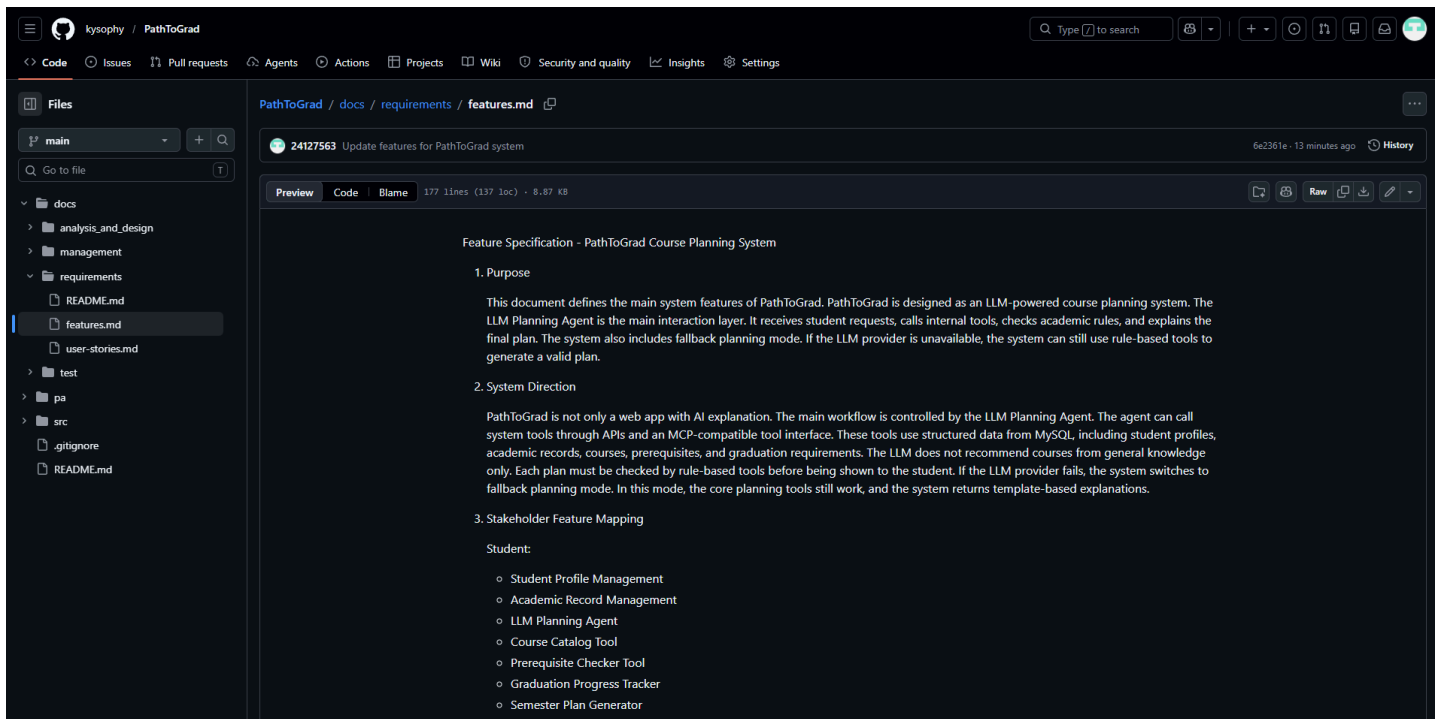
Task 3: Write Features Documentation

Description: Supported the preparation of Section 3.1.1 Features in the project proposal. Organized the feature content into stakeholder-feature mapping, user story mapping summary, main features, LLM-as-main-program workflow, tools, APIs, skills, requirements, and constraints.

- **Visual Evidence:**



Created user stories on Github.



Created features on Github.

Stake-holder feature mapping

Stakeholder	Main Need	Related Features
Student	Plan semesters, check prerequisites, track graduation progress, and understand academic risks.	Student Profile Management, Academic Record Management, LLM Planning Agent, Course Catalog Tool, Prerequisite Checker Tool, Graduation Progress Tracker, Semester Plan Generator, Academic Risk Detector, AI Explanation and Warning, Fallback Planning Mode
Academic Advisor	Review student plans and identify possible academic risks.	Advisor Plan Review View, Advisor Summary Skill
Academic Staff / Admin	Maintain course data, prerequisite rules, and curriculum requirements.	Course Data Management, Course Data Import API

Write features documentation.

24127080 - Ông Khánh Minh - 25%

- **Task Evidence:**

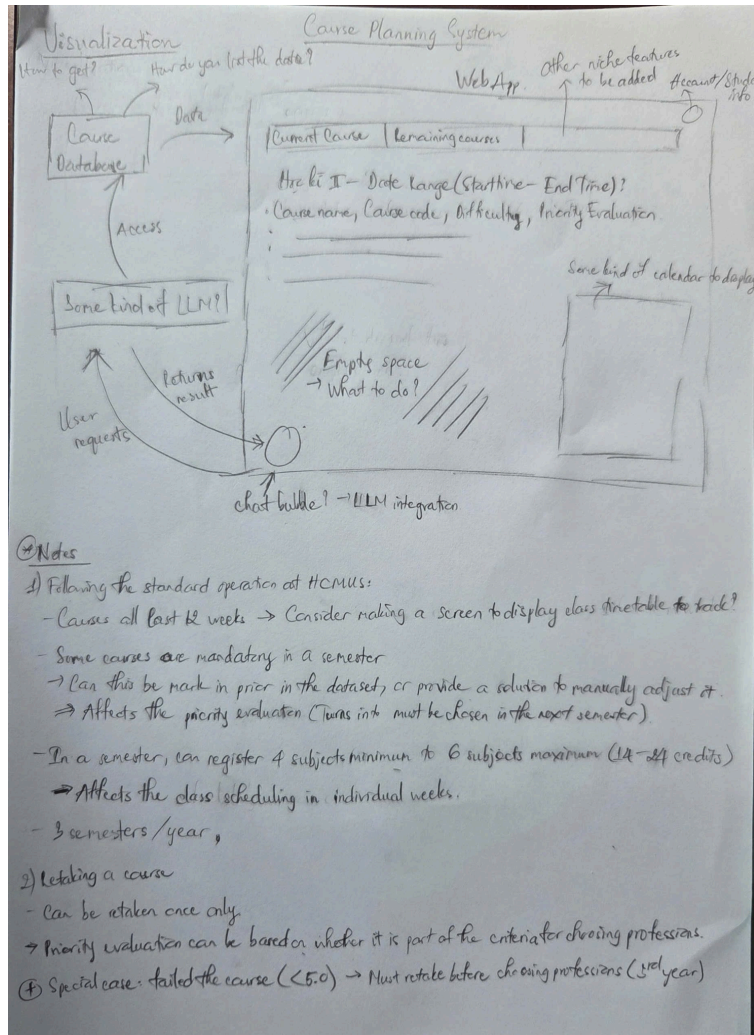
Task 1: Software Architecture

Description: Designed the comprehensive system blueprint for PathToGrad, establishing a decoupled, four-layer architecture (Presentation, Web Router, Orchestration, and Core Logic). Engineered the data flow to integrate an LLM as the main operational agent while enforcing strict deterministic fallbacks for university constraints. Visualized structural flowcharts illustrating the AI routing, API integrations, and manual bypass pathways.

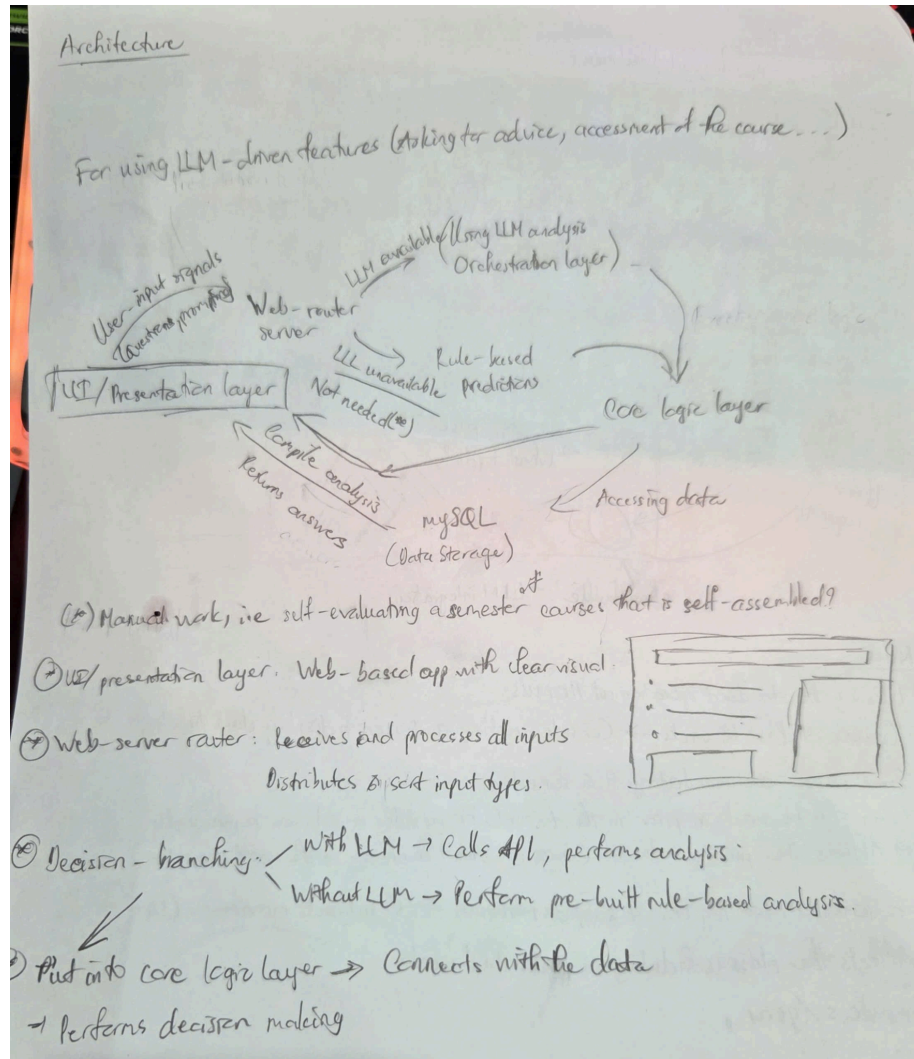
Task 2: Hardware discussion

Description: Defined the computational infrastructure required across client, server, and development environments. Formulated the strategic hardware rationale for offloading heavy AI processing to cloud-hosted APIs (OpenAI/Gemini), successfully eliminating the need for expensive local GPU servers and ensuring the application remains lightweight, cost-effective, and highly deployable on standard web hosting.

- **Visual Evidence:**



First drafting of the interface -> Visualization for the architecture



A more comprehensive work on the architecture

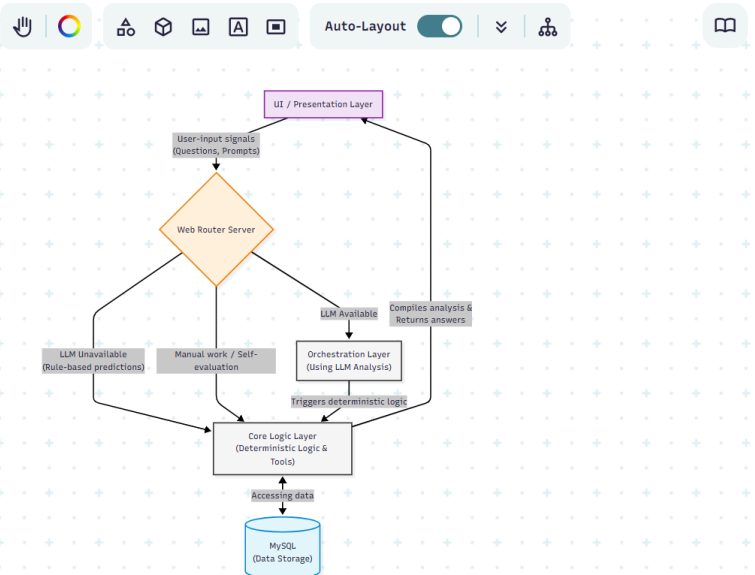
Personal files / Web Router Server

Export Share 15 Credits Start my trial

```

1 flowchart TD
2
3   UI["UI / Presentation Layer"]
4   Router{"Web Router Server"}
5   LLM["Orchestration Layer<br/>(Using LLM AI)"]
6   Logic["Core Logic Layer<br/>(Deterministic Logic)"]
7   DB["MySQL<br/>(Data Storage)"]
8
9
10  UI -->|"User-input signals<br/>(Questions, Prompts)"| Router
11
12  %% Processing paths (LLM usage)
13  Router -->|"LLM Available"| LLM
14  Router -->|"LLM Unavailable<br/>(Rule-based predictions)"| Logic
15  Router -->|"Manual work / Self-evaluation"| Logic
16
17  LLM -->|"Triggers deterministic logic"| Logic
18
19  %% Database Interaction
20  Logic <-->|"Accessing data"| DB
21
22  %% Return to interface
23  Logic -->|"Compiles analysis & Returns answers"| UI
24
25  classDef default fill:#f9f9f9,stroke:#333,stroke-width:1px
26  classDef db fill:#e1f5fe,stroke:#0288d1,stroke-width:1px
27  classDef router fill:#fff3e0,stroke:#f57c00,stroke-width:1px
28  classDef ui fill:#f3e5f5,stroke:#8e24aa,stroke-width:1px
29  classDef llm fill:#e8f5e9,stroke:#4caf50,stroke-width:1px
30
31  class Router router
32  class UI ui
33  class LLM llm
34  class Logic default
35  class DB db

```



Crafting diagram visualization for better clarity

2 Preliminary Problem Statement

Written by: 24127543 - Nguyễn Trương Ngọc Thảo

Edited by:

Reviewed by: 24127563 - Trần Thị Ngọc Trâm

2.1 Introduction

In order to graduate, students are required to finish a sequence of courses scattered throughout every semester over four years. During this time, the demand for clear instructions regarding semester planning, schedule clarity, and prerequisite chains is strict and requires an enormous amount of time.

Manually arranging schedules using scattered PDF files, various websites, disjointed catalogs, and spreadsheets is prone to human error. Some students cannot schedule the correct courses logically or track their academic results and remaining compulsory courses. This sometimes leads to delayed graduation timelines, which places an unnecessary financial burden on students and negatively impacts the university's graduation rate.

The objective of this software is to build a centralized platform for students to plan appropriate courses according to their current scores and remaining compulsory requirements, ultimately clearing a path to graduation. This solution will provide an interface where students can check their current semester's schedule, scores, and available courses. It will also utilize an LLM to arrange and manage everything, from checking academic history and calculating course results to recommending a new course plan.

2.2 Operating environment

The system will operate as a web-based application to ensure maximum accessibility for the student body and administrative staff.

- **Client Interface:** The application will be accessed via web browsers. The backend system will be based on Python and FastAPI or [Node.js](#) and Express. Frontend will

mainly use React and Vite. Programming language will be Typescript.

- **Server Environment:** The application will be hosted on a standard cloud-based server environment capable of handling high access during registration weeks.
- **Future Enhancement:** The system shall be designed to support future secure, read-only integration with the university Registrar and Student Information Systems. In the current version, course and academic data may be prepared manually or imported through CSV/MySQL seed scripts by the development team.

2.3 Design & Implementation Constraints

To ensure maintainability and consistency, the design and implementing progress must follow the listed constraints:

- **Technical:** The system will use Python for the backend to handle the complex scheduling logic, and MySQL to safely store course information and student records instead of json files.
- **Data:** Based on MySQL. The system's logic must follow the existing university rules. It can not allow students to register or generate a plan that conflicts with university's constraints, ensuring a clear schedule. The generated data will attempt to follow the official course data as close as possible.
- **LLM/AI:** OpenAI/Gemini API.
- **Document:** All source code must be documented according to advisors's software engineering practices.

3

Proposed Solution

3.1 Software

3.1.1. Features

Written by: 24127563 - Trần Thị Ngọc Trâm

Edited by:

Reviewed by:

This section defines the main features of PathToGrad, an LLM-powered course planning system for university students. The features are based on stakeholder needs, user story mapping, and the course planning problems described in the preliminary problem statement.

PathToGrad focuses on helping students plan semesters, check prerequisite chains, track graduation progress, and understand academic risks. The system is not only a normal course catalog or a chatbot. It is designed with the LLM as a main program. This means that students interact with the LLM Planning Agent first, and the agent coordinates system tools to complete the planning task.

The LLM Planning Agent does not make course plans from general knowledge only. It uses structured data from the system, including student profiles, academic records, course catalogs, prerequisite rules, and graduation requirements. The agent calls internal tools such as the Course Catalog Tool, Prerequisite Checker Tool, Graduation Progress Tracker, Semester Plan Generator, and Academic Risk Detector.

To improve reliability, the system also includes a fallback planning mode. If the LLM provider is unavailable, the system can still run the rule-based planning tools and return a valid plan with template-based explanations. Therefore, the core planning features do not fully depend on one external AI provider.

Stakeholder-Feature mapping

Stakeholder	Main Need	Related Features
Student	Plan semesters, check prerequisites, track graduation progress, and understand academic risks.	Student Profile Management, Academic Record Management, LLM Planning Agent, Course Catalog Tool, Prerequisite Checker Tool, Graduation Progress Tracker, Semester Plan Generator, Academic Risk Detector, AI Explanation and Warning, Fallback Planning Mode
Academic Advisor	Review student plans and identify possible academic risks.	Advisor Plan Review View, Advisor Summary Skill
Academic Staff / Admin	Maintain course data, prerequisite rules, and curriculum requirements.	Course Data Management, Course Data Import API

User story mapping summary

User Activity	User Need	User Story	Related Feature
Create academic profile	Students need the system to understand their curriculum context.	As a student, I want to create my academic profile, so that the system can understand my major, intake year, semester, and credit target.	F-01
Enter academic record	Students need the system to know completed and failed courses.	As a student, I want to enter my academic record, so that the system can identify completed, failed, and remaining courses.	F-02
Ask for planning support	Students need a smart assistant for course planning.	As a student, I want to ask the LLM Planning Agent for a semester plan, so that I can receive a suitable and explainable recommendation.	F-03
Retrieve course data	The agent needs accurate course data before planning.	As a student, I want the system to use course catalog data, so that the plan is based on valid course information.	F-04

Check prerequisites	Students need to avoid invalid course choices.	As a student, I want the system to check prerequisites, so that I only choose courses that match my academic record.	F-05
Track graduation progress	Students need to know completed and missing requirements.	As a student, I want to track my graduation progress, so that I can know what I still need to complete.	F-06
Generate semester plan	Students need a valid plan for the next semester.	As a student, I want the system to suggest a semester plan, so that I can choose suitable courses.	F-07
Detect academic risks	Students need warnings before making risky decisions.	As a student, I want the system to detect academic risks, so that I can avoid invalid choices or delayed graduation.	F-08
Understand recommendation	Students need to understand the reason behind the plan.	As a student, I want to see explanations and warnings, so that I can understand the suggested plan.	F-09
Use fallback planning	Students need the system to remain usable when the LLM provider is unavailable.	As a student, I want the system to provide a fallback plan, so that I can still use the core planning features.	F-10
Review student plan	Advisors need to review student plans and risks.	As an academic advisor, I want to view a student's plan summary, so that I can give suitable academic advice.	F-11
Manage course data	Academic staff need to keep course and prerequisite data correct.	As an academic staff member, I want to update course and prerequisite data, so that the system can generate accurate course plans.	F-12

Main Features:

Feature ID	Feature Name	Main User	Description
F-01	Student Profile Management	Student	Allows students to create and update their profile, including major, intake year, current semester, and target credit load.
F-02	Academic Record Management	Student	Allows students to enter completed courses, failed courses, grades, and credits. This data is used by the planning agent and internal tools.
F-03	LLM Planning Agent	Student	Acts as the main interaction layer of the system. It receives student requests, selects suitable tools, calls APIs, and explains the final result.
F-04	Course Catalog Tool	LLM Agent/ Student	Retrieves course information from MySQL, including course code, course name, credits, suggested semester, and prerequisites.
F-05	Prerequisite Checker Tool	LLM Agent/ Student	Checks whether a student satisfies prerequisite rules before a course is recommended
F-06	Graduation Progress Tracker	LLM Agent/ Student	Calculates completed credits, remaining credits, completed required courses, and missing required courses.
F-07	Semester Plan Generator	LLM Agent/ Student	Suggests suitable courses for the next semester based on academic record, prerequisite rules, credit load, and graduation requirements.
F-08	Academic Risk Detector	LLM Agent/ Student	Detects risks such as missing prerequisites, overloaded study plans, delayed required courses, or missing graduation requirements.
F-09	AI Explanation and Warning	Student	Explains why each course is recommended and shows warnings in clear language.
F-10	Fallback Planning Mode	Student	Allows the system to continue core planning functions when the LLM provider is unavailable. It uses rule-based tools and template-based explanations.
F-11	Advisor Plan Review View	Academic Advisor	Allows advisors to view a student's semester plan, warnings, and graduation progress summary.
F-12	Course Data Management	Academic Staff/ Admin	Allows academic staff or admin users to manage course and prerequisite data through MySQL seed scripts, CSV

			import, or admin APIs.
F-13	API Layer	Frontend/ LLM Agent	Provides backend endpoints for student data, course data, planning requests, tool execution, advisor view, and admin data updates.
F-14	MCP-compatible Tool Interface	LLM Agent	Defines structured tool names, inputs, and outputs so that the LLM Planning Agent can call tools in a clear way.
F-15	Planning Skills	LLM Agent	Provides reusable workflows such as Plan Next Semester, Validate Study Plan, Review Graduation Progress, and Advisor Summary.
F-16	Agent Run Log	System/ Admin	Records agent runs, tool calls, provider errors, and fallback actions for review and debugging.

LLM as main program workflow

The main workflow is organized around the LLM Planning Agent:

1. The student sends a planning request to the LLM Planning Agent.
2. The agent reads the student profile and academic record.
3. The agent calls the Course Catalog to retrieve course data.
4. The agent calls the Prerequisite Checker Tool to remove invalid course options.
5. The agent calls the Graduation Progress Tracker to identify missing requirements.
6. The agent calls the Semester Plan Generator to create a semester plan.
7. The agent calls the Academic Risk Detector to detect possible risks.
8. The agent explains the final plan and warning to the student.

If The LLM provider is unavailable, the system switches to fallback planning mode. In this mode, the same rule-base tools are still used, but the explanation is generated from prepared templates.

Tools, APIs, MCP-compatible Interface and Skills

Type	Name	Purpose
Tool	Course Catalog Tool	Gets course information from MySQL.

Tool	Prerequisite Checker Tool	Checks whether a student can take a course.
Tool	Graduation Progress Tracker	Calculates completed and missing graduation requirements.
Tool	Semester Plan Generator	Creates a recommended semester plan.
Tool	Academic Risk Detector	Finds academic risks in a plan.
API	Student API	Manages student profile and academic records.
API	Course API	Provides course and prerequisite data.
API	Agent API	Receives planning requests and returns agent responses.
API	Tool API	Allows the agent to call internal tools.
API	Admin API	Supports course data import and updates.
MCP-compatible Interface	Planning Tool Interface	Defines tool names, input schema, output schema, and tool status.
Skill	Plan Next Semester	Uses tools to create a semester plan.
Skill	Validate Study Plan	Checks selected courses, prerequisites, and credit load.
Skill	Review Graduation Progress	Reviews completed credits, missing courses, and risks.
Skill	Advisor Summary	Produces a short planning summary for advisors.

Functional requirements

ID	Functional Requirement	Related Feature
FR-01	The system shall allow students to create and update their academic profile.	F-01
FR-02	The system shall allow students to enter completed courses, failed courses, grades, and credits.	F-02
FR-03	The system shall allow students to send planning requests to the LLM Planning Agent.	F-03
FR-04	The system shall allow the LLM Planning Agent to call internal tools through APIs.	F-03, F-13
FR-05	The system shall provide an MCP-compatible tool interface with structured tool inputs and outputs.	F-14
FR-06	The system shall retrieve course data from MySQL.	F-04
FR-07	The system shall check whether a student satisfies prerequisite rules.	F-05
FR-08	The system shall calculate graduation progress.	F-06
FR-09	The system shall generate a recommended semester plan.	F-07
FR-10	The system shall detect academic risks in a study plan.	F-08
FR-11	The system shall explain the recommended plan in clear language.	F-09
FR-12	The system shall show warnings when a plan contains academic risks.	F-09
FR-13	The system shall provide fallback planning mode when the LLM provider is unavailable.	F-10
FR-14	The system shall generate template-based explanations in fallback mode.	F-10
FR-15	The system shall allow academic advisors to view plan summaries and warnings.	F-11
FR-16	The system shall allow academic staff or admin users to update course and prerequisite data.	F-12
FR-17	The system shall record agent runs, tool calls, provider errors, and fallback actions.	F-16

Non-functional requirements

ID	Non-functional Requirement
NFR-01	The system should be accessible through modern web browsers.
NFR-02	The system should present planning results in clear and simple language.
NFR-03	The system should protect student academic information.
NFR-04	The system should restrict access based on user roles.
NFR-05	The system should store course, prerequisite, student, and academic record data in MySQL.
NFR-06	The system should verify LLM-generated recommendations using rule-based tools.
NFR-07	The system should continue core planning functions when the LLM provider is unavailable.
NFR-08	The system should show clear warnings when data is missing, outdated, or uncertain.
NFR-09	The system should log important agent actions and tool calls for traceability.

Constraints

The system depends on available course data, prerequisite rules, academic records, and curriculum requirements. In the current version, the data may be prepared manually or imported from CSV/MySQL seed scripts.

The system recommendations are based on the available data in the database. If some course data, prerequisite rules, or curriculum requirements are missing or uncertain, the system should show warnings to users.

The LLM Planning Agent must not recommend courses from unsupported general knowledge. All course recommendations must be checked by internal tools before being shown to the student.

If the LLM provider fails, the system should switch to fallback planning mode and continue to provide rule-based planning results. In fallback planning mode, the system will not generate AI explanations. Instead, it will use template-based explanations based on tool results.

The system supports academic planning only. Official course registration is outside the

current scope.

The MCP-compatible interface defines how the LLM calls tools. A full external MCP server can be added in a later version.

Course data can be managed through MySQL seed scripts, CSV import, or admin APIs.

Real-time integration with the university registrar system is considered a future enhancement.

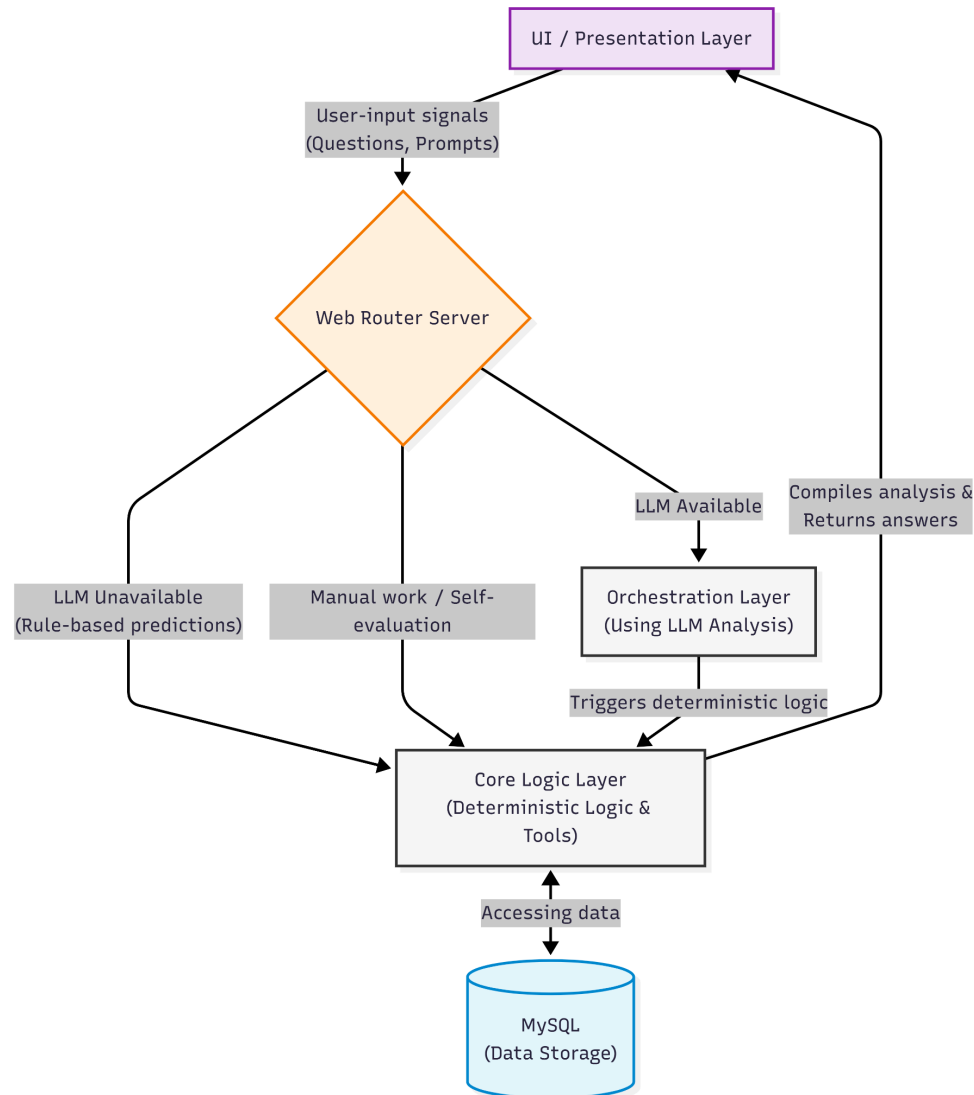
3.1.2. Software Architecture

Written by: 24127080 - Ông Khánh Minh

Edited by:

Reviewed by:

Following the agentic workflow and strict academic guardrails presented in this project, the system implements a Decoupled Layered Architecture with an Agentic Tool-Execution Loop. The system is split into four distinct layers to ensure separation of concerns, high reliability, and a predictable fallback state if the external LLM provider is unreachable



Presentation Layer (Frontend)

The user interface provides a dual-view workspace: a conversational chat bubble for natural language interactions with the LLM Planning Agent, and a synchronized 7-day calendar scheduler that visually renders weekly theory (LT) and laboratory (TH) time slots. It captures user text prompts and package changes, transmitting them as structured JSON payloads to the backend API.

Web Router Layer (FastAPI Application Server)

Acting as the central gateway, this layer receives all frontend requests. It handles user authentication, session state management, and crucially coordinates the system's

operational mode. It actively monitors outbound connections to the external LLM API. If a timeout or provider error is intercepted, it automatically reroutes the traffic to the Fallback Controller, maintaining core application functionality without crashes.

Orchestration & Fallback Layer

- LLM Planning Agent Mode (Default): The main program runtime. It parses the student's natural language intent, accesses the available tools via an MCP-compatible interface, and dynamically generates action plans based strictly on returned tool data.
- Fallback Planning Mode: A rule-based backup subsystem. If the AI provider fails, this component directly calls the core logic tools and returns structural planning data formatted through static text templates, guaranteeing zero downtime for critical features.

Core Logic Layer

The LLM is strictly isolated from calculating rules to eliminate hallucinations. This layer consists of pure, deterministic Python algorithms that execute specific verification scripts:

- Prerequisite Checker Tool: Represents the course catalog as a Directed Acyclic Graph (DAG), traversing nodes via algorithms to validate that all prerequisites are cleared based on the student's academic record.
- Course Scheduling Engine: Stems from current logics applied, including the 3-semesters-per-year schedule rhythm, strict credit minimums and maximums (14 to 24 credits per semester, or 4 to 6 courses), weekly class code time slot conflicts, and an enforcement filter forcing the prioritized retaking of failed courses (grades < 5.0) prior to the 3rd-year specialization window.

Data Layer (MySQL Database)

A relational database system managing the persistent application state. It maps out structured schemas for:

- Tracks basic academic profiles, intake years, and major contexts.
- Tracks passed, in-progress, and failed courses along with earned credits.
- Holds definitive university course metadata, language configurations (e.g. English flags for "Cơ sở ngành" course, which indicates this), specific class module time matrices (Theory vs. Lab periods), and explicit prerequisite relationship links.

3.2 Hardware

Written by: 24127080 - Ông Khánh Minh

Edited by:

Reviewed by:

PathToGrad is designed as a lightweight web application that delegates heavy artificial intelligence processing to cloud-based APIs (e.g., OpenAI or Gemini). Consequently, the system does not require specialized hardware, such as dedicated GPUs, for local or server-side execution. The hardware requirements are divided into three operational environments:

Client-side Requirements (End-User)

The application is accessed via a web browser. The student's device only needs sufficient resources to render the React frontend and handle standard JavaScript execution.

- Device: Any modern desktop, laptop, tablet, or smartphone.
- Processor (CPU): 1.0 GHz dual-core processor or higher.
- Memory (RAM): Minimum 2 GB.
- Network: Stable internet connection for real-time chat and scheduling updates.
- Software: A modern, updated web browser (Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge).

Server-side Requirements (Hosting & Production)

To host the compiled React (Vite) frontend, the Backend application (Node.js/Express or Python/FastAPI), and the Database (SQLite/MySQL), the deployment server requires the following minimum specifications:

- Processor (vCPU): Minimum 2 Cores (e.g., standard cloud compute instance like AWS t3.micro or equivalent).
- Memory (RAM): Minimum 4 GB RAM to comfortably handle concurrent API requests, tool execution, and database queries.
- Storage: 20 GB SSD (Solid State Drive) to ensure fast read/write speeds for the database and to store application logs/source code.
- Network Interface: High-bandwidth connection to ensure low latency when the backend communicates with the external LLM provider APIs.
- GPU: None required (AI processing is offloaded to the external API provider).

Development Environment Requirements (Team)

To develop, test, and run the full stack locally (Frontend server, Backend server, and Database instance), developers require standard computing environments.

- Processor: Intel Core i5 / AMD Ryzen 5 or equivalent.
- Memory (RAM): 8 GB RAM (16 GB recommended for running multiple Docker containers or heavy IDEs concurrently).
- Storage: 50 GB of free space, preferably on an SSD.
- Operating System: Windows 10/11, macOS, or Linux.

4

Development Plan

Written by: 24127427 - Trần Nhật Đăng Khoa

Edited by:

Reviewed by: 24127427 - Trần Nhật Đăng Khoa

The PathToGrad project will follow a phased software development approach. Each phase focuses on a specific set of activities and produces deliverables that serve as input for subsequent.

4.1 Requirements Analysis

Activities

- Identify stakeholders.
- Analyze course planning challenges faced by students.
- Gather and document system requirements.
- Create user stories and user story maps.
- Define functional and non-functional requirements.

Deliverables

- Preliminary Problem Statement
- Vision Document
- User Stories
- User Story Map
- Requirements Specification

4.2 *Software Design*

Activities

- Design the overall software architecture.
- Design the database structure.
- Create UML diagrams.
- Design user interface mockups.
- Define interactions between system components.

Deliverables

- Software Architecture Document
- UML Diagrams
- Database Design
- UI Mockups

4.3 *Implementation*

Activities

- Develop frontend components.
- Develop backend services.
- Implement database functionality.
- Integrate system components.

Deliverables

- Source Code
- Executable Prototype
- Database Scripts

4.4 *Testing*

Activities

- Create test plans and test cases.

- Perform functional testing.
- Identify and fix defects.
- Verify system requirements.

Deliverables

- Test Plan
- Test Cases
- Test Reports
- Bug Fix Reports

4.5 *Deployment and Maintenance****Activities***

- Deploy the application.
- Monitor system performance.
- Fix reported issues.
- Maintain course data and system configurations.
- Implement minor enhancements and updates.

Deliverables

- Deployed System
- User Documentation
- Maintenance Reports
- Final Project Release

5

Human Resources & Costing Plans

Written by: 24127427 - Trần Nhật Đăng Khoa

Edited by:

Reviewed by:

5.1 Human Resources Plan

The **PathToGrad** project will be developed by a team of four members. Responsibilities are distributed according to each member's role to ensure effective project execution.

Role	Responsibilities
Project Manager	Project planning, task assignment, progress monitoring, document integration, development planning, and team coordination.
Business Analyst	Problem analysis, stakeholder identification, research, and preparation of the Preliminary Problem Statement.
Requirements Analyst	User story mapping, feature specification, functional requirements analysis, and requirement documentation.
System Architect	Software architecture design, hardware requirements analysis, technical planning, and system structure documentation.

5.2 Development Phase Allocation

Phase	Responsible Roles
Requirements Analysis	Business Analyst, Requirements Analyst
Software Design	System Architect, Requirements Analyst
Implementation	Entire Team
Testing	Entire Team
Deployment & Maintenance	Entire Team

5.3 Costing Plan

The project primarily relies on free educational and open-source tools. Therefore, the anticipated project cost is minimal.

Resource	Purpose	Estimated Cost
GitHub	Version control and documentation storage	Free
Jira	Project management and sprint tracking	Free (Educational/Free Plan)
Discord	Team communication and collaboration	Free
Moodle	Assignment submission and course communication	Provided by the University
Development Tools (IDE, Browser, Database)	Software development and testing	Free
LLM API (OpenAI/Gemini)	AI planning assistant	Free tier during development

Total Estimated Cost

The estimated direct project cost is **\$0 USD** as all required tools and resources are available through free plans or university-provided services.

6

Tools setup

Written by: 24127427 - Trần Nhật Đăng Khoa

Edited by:

Reviewed by:

To support effective collaboration and project management throughout the development of **PathToGrad**, the team will utilize the following tools:

Moodle

Moodle will be used as the official platform for receiving project requirements, announcements, and submitting project deliverables throughout the semester.

Discord

The team will use Discord as its primary communication platform. Dedicated channels will be created for project discussions, announcements, meeting coordination, and technical support. Team members are expected to actively participate and regularly check project updates.

Jira

Jira will be used for project planning and task management. Tasks will be organized into sprints corresponding to project milestones and assignment deadlines. Each task will be assigned to a specific team member and tracked through its lifecycle to ensure accountability and transparency.

GitHub

GitHub will be used for version control and project documentation management. The repository will store all project artifacts and maintain revision history throughout development.

The repository structure is organized as follows:

- `/src` – Stores source code.
- `/docs/management` – Stores planning documents, progress reports, and project management artifacts.
- `/docs/requirements` – Stores requirements-related documents, including vision documents and use cases.
- `/docs/analysis_and_design` – Stores software architecture documents, UML diagrams, UI designs, and other design artifacts.
- `/docs/test` – Stores test plans, test cases, and test reports.
- `/pa` – Stores assignment submissions for each project phase.
- Link: <https://github.com/kysophy/PathToGrad>

Team Collaboration Process

The team will use **Discord** for communication, **Jira** for task tracking, **GitHub** for version control and document management, and **Moodle** for official course submissions. Regular updates will be maintained across all platforms to ensure effective collaboration and transparency among team members.

7

AI Usage Declaration

Based on the provided AI Usage Guideline document, teams must submit their AI usage declaration here. Students are encouraged to leverage AI tools creatively and effectively in this project.

8

Presentation

Teams are required to record presentation videos of their work using the provided template.

Every team member must participate in the presentation, specifically covering the sections they contributed to the project.

Videos must not exceed 30 minutes.

Please upload your videos to Youtube (set to either Unlisted or Public).

Provide the Youtube links to the field below.

9

Reflective Report

Which sections of this template are most helpful for implementation, and which are unnecessary? Provide specific examples to support your reasoning.